

Meep Tutorials

Advisor:-

Prof. Sayak Bhattacharya, IIIT Delhi

Presentation by:-

Chinmaya Sharma, B.Tech. EVE '28, IIIT Delhi



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**



Tutorials performed



- Fields in a Waveguide: A Straight Waveguide
- Fields in a Waveguide: A 90° Bend
- Fields in a Waveguide: Transmittance Spectrum of a Waveguide Bend
- Angular Reflectance Spectrum of a Planar Interface
-
- SoC Platforms
- Caravel
- Caravel Datasheet

Installing Miniconda



```
chinmaya24163@DESKTOP-79DPGGQ:~$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
--2026-05-17 21:19:08-- https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
Resolving repo.anaconda.com (repo.anaconda.com)... 104.16.191.158, 104.16.32.241, 2606:4700::6810:20f1, ...
Connecting to repo.anaconda.com (repo.anaconda.com)|104.16.191.158|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 163179296 (156M) [application/octet-stream]
Saving to: 'Miniconda3-latest-Linux-x86_64.sh'
```

```
Miniconda3-latest-Linux-x86_64 100%[=====>] 155.62M  24.4MB/s   in 6.7s
2026-05-17 21:19:15 (23.1 MB/s) - 'Miniconda3-latest-Linux-x86_64.sh' saved [163179296/163179296]
```

```
chinmaya24163@DESKTOP-79DPGGQ:~$ bash Miniconda3-latest-Linux-x86_64.sh
```

```
Welcome to Miniconda3 py313_26.3.2-2
```

```
In order to continue the installation process, please review the license
agreement.
```

```
Please, press ENTER to continue
```

```
>>>
```

```
MINICONDA END USER LICENSE AGREEMENT
```

```
Copyright Notice: Miniconda(R) (C) 2015, Anaconda, Inc.
All rights reserved. Miniconda(R) is licensed, not sold.
```

```
Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
```

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer;
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution;
3. The name Anaconda, Inc. or Miniconda(R) may not be used to endorse or promote products derived from this software without specific prior written permission from Anaconda, Inc.; and
4. Miniconda(R) may not be used to access or allow third parties to access Anaconda package repositories if such use would circumvent paid licensing requirements or is otherwise restricted by the Anaconda Terms of Service.

```
DISCLAIMER: THIS SOFTWARE IS PROVIDED BY ANACONDA "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT ARE DISCLAIMED.
```

Creating a Meep environment



```
(base) chinmaya24163@DESKTOP-790PGGQ:~$ conda create -n mp -c conda-forge pymeep
Do you accept the Terms of Service (ToS) for https://repo.anaconda.com/pkg/main? [(a)accept/(r)reject/(v)view]: a
Do you accept the Terms of Service (ToS) for https://repo.anaconda.com/pkg/r? [(a)accept/(r)reject/(v)view]: a
2 channel Terms of Service accepted
Retrieving notices: done
Channels:
- conda-forge
- defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /home/chinmaya24163/miniconda3/envs/mp

added / updated specs:
- pymeep

The following packages will be downloaded:
```

package	build	
openmp_mutex-4.5	20_gnu	28 KB conda-forge
brotil-1.2.0	hed03a55_1	20 KB conda-forge
brotil-bin-1.2.0	hb03c661_1	21 KB conda-forge
bzip2-1.0.8	hda65f42_9	254 KB conda-forge
c-ares-1.34.6	hb03c661_0	203 KB conda-forge
ca-certificates-2026.4.22	hbd8a1cb_0	128 KB conda-forge
cached-property-1.5.2	hd8ed1ab_1	4 KB conda-forge
cached-property-1.5.2	pyha778c72_1	11 KB conda-forge
contourpy-1.3.3	py313hc8edb43_4	314 KB conda-forge
cycler-0.12.1	pyhcf101f3_2	14 KB conda-forge
fftw-3.3.11	nompi_h3b011a4_100	2.1 MB conda-forge
fonttools-4.63.0	py313h3dea7bd_0	2.9 MB conda-forge
freetype-2.14.3	ha778c72_0	170 KB conda-forge
gst-2.7	hea38d99_0	3.2 MB conda-forge
h5py-3.16.0	nompi_py313h253c126_102	1.3 MB conda-forge
harminv-1.4.3	h63ef21f_0	49 KB conda-forge
hd5-1.14.6	nompi_h1948ede_109	3.5 MB conda-forge
keyutils-1.6.3	hb9d3cd8_0	131 KB conda-forge
kiwisolver-1.5.0	py313hc8edb43_0	75 KB conda-forge
krb5-1.22.2	ha1258a1_0	1.3 MB conda-forge
lcms2-2.19.1	h0c27ade_0	245 KB conda-forge
ld_impl_linux-64-2.45.1	default_hbd61a6d_102	711 KB conda-forge
lerc-4.1.0	hdb60285_0	255 KB conda-forge
libaec-1.1.5	h088129d_0	36 KB conda-forge
libblas-3.11.0	7_h4a7c4f45_openblas	18 KB conda-forge
libbrotlicommon-1.2.0	hb03c661_1	78 KB conda-forge
libbrotlidec-1.2.0	hb03c661_1	34 KB conda-forge
libbrotlienc-1.2.0	hb03c661_1	291 KB conda-forge
libblas-3.11.0	7_h0358290_openblas	18 KB conda-forge
libctf-4.6.0	hb03c661_0	111 KB conda-forge
libcurl-8.20.0	hcf29cc6_0	458 KB conda-forge
libdeflate-1.25	h17f619e_0	72 KB conda-forge
libedit-3.1.20250104	pl5321h7949ede_0	122 KB conda-forge
libev-4.33	hds90300_2	110 KB conda-forge
libexpat-2.8.0	hecca717_0	75 KB conda-forge
libffi-3.5.2	h3435931_0	57 KB conda-forge
libfreetype-2.14.3	ha778c72_0	8 KB conda-forge
libfreetype6-2.14.3	h73754d4_0	376 KB conda-forge
libgcc-15.2.0	he0febe6_19	1017 KB conda-forge
libgcc-ng-15.2.0	h69a702a_19	27 KB conda-forge

Fields in a Waveguide: A Straight Waveguide



```
GNU nano 6.2
# From the Meep tutorial: plotting permittivity and fields of a straight waveguide
import meep as mp

cell = mp.Vector3(16, 8, 0)

geometry = [
    mp.Block(
        mp.Vector3(mp.inf, 1, mp.inf),
        center=mp.Vector3(),
        material=mp.Medium(epsilon=12),
    )
]

sources = [
    mp.Source(
        mp.ContinuousSource(frequency=0.15), component=mp.Ez, center=mp.Vector3(-7, 0)
    )
]

pml_layers = [mp.PML(1.0)]

resolution = 10

sim = mp.Simulation(
    cell_size=cell,
    boundary_layers=pml_layers,
    geometry=geometry,
    sources=sources,
    resolution=resolution,
)

sim.run(until=200)

import matplotlib.pyplot as plt
import numpy as np

eps_data = sim.get_array(center=mp.Vector3(), size=cell, component=mp.Dielectric)
plt.figure()
plt.imshow(eps_data.transpose(), interpolation="spline36", cmap="binary")
plt.axis("off")
plt.show()

ez_data = sim.get_array(center=mp.Vector3(), size=cell, component=mp.Ez)
plt.figure()
plt.imshow(eps_data.transpose(), interpolation="spline36", cmap="binary")
plt.imshow(ez_data.transpose(), interpolation="spline36", cmap="RdBu", alpha=0.9)
plt.axis("off")
plt.show()
```

Fields in a Waveguide: A Straight Waveguide

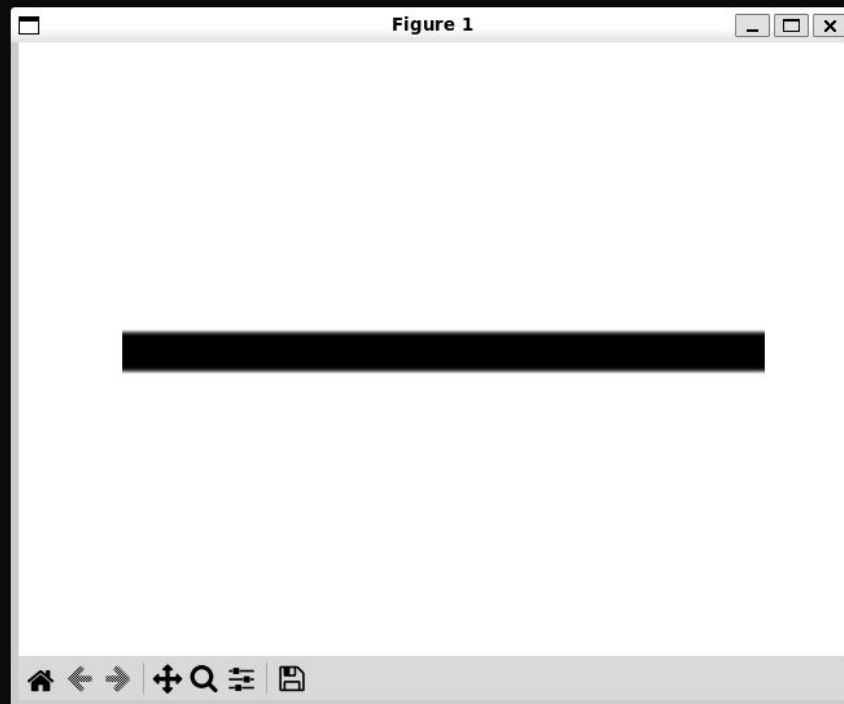


```
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ python straight_waveguide.py
```

```
-----
Initializing structure...
time for choose_chunkdivision = 0.0003438 s
Working in 2D dimensions.
Computational cell is 16 x 8 x 0 with resolution 10
  block, center = (0,0,0)
  size (1e+20,1,1e+20)
  axes (1,0,0), (0,1,0), (0,0,1)
  dielectric constant epsilon diagonal = (12,12,12)
time for set_epsilon = 0.010875 s
-----
run 0 finished at t = 200.0 (4000 timesteps)
```

```
Elapsed run time = 40.6971 s
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ python straight_waveguide.py
```

```
-----
Initializing structure...
time for choose_chunkdivision = 8.2016e-05 s
Working in 2D dimensions.
Computational cell is 16 x 8 x 0 with resolution 10
  block, center = (0,0,0)
  size (1e+20,1,1e+20)
  axes (1,0,0), (0,1,0), (0,0,1)
  dielectric constant epsilon diagonal = (12,12,12)
time for set_epsilon = 0.010633 s
-----
run 0 finished at t = 200.0 (4000 timesteps)
```

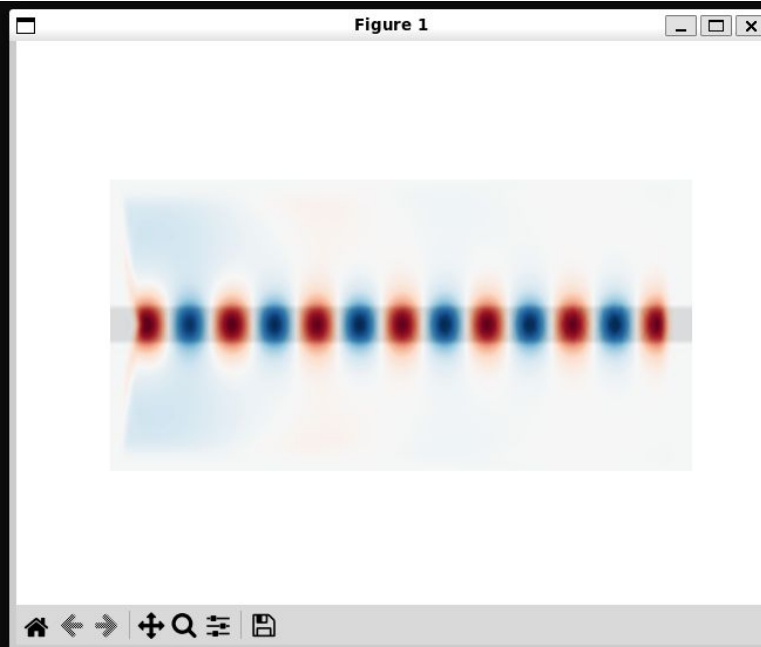


Fields in a Waveguide: A Straight Waveguide



```
(mp) chinmaya24163@DESKTOP-79DPGGQ: ~/meep_tutorials$ python straight_waveguide.py
-----
Initializing structure...
time for choose_chunkdivision = 0.0003438 s
Working in 2D dimensions.
Computational cell is 16 x 8 x 0 with resolution 10
  block, center = (0,0,0)
  size (1e+20,1,1e+20)
  axes (1,0,0), (0,1,0), (0,0,1)
  dielectric constant epsilon diagonal = (12,12,12)
time for set_epsilon = 0.010875 s
-----
run 0 finished at t = 200.0 (4000 timesteps)

Elapsed run time = 40.6971 s
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ python straight_waveguide.py
-----
Initializing structure...
time for choose_chunkdivision = 8.2016e-05 s
Working in 2D dimensions.
Computational cell is 16 x 8 x 0 with resolution 10
  block, center = (0,0,0)
  size (1e+20,1,1e+20)
  axes (1,0,0), (0,1,0), (0,0,1)
  dielectric constant epsilon diagonal = (12,12,12)
time for set_epsilon = 0.010633 s
-----
run 0 finished at t = 200.0 (4000 timesteps)
```



Fields in a Waveguide: A 90° Bend



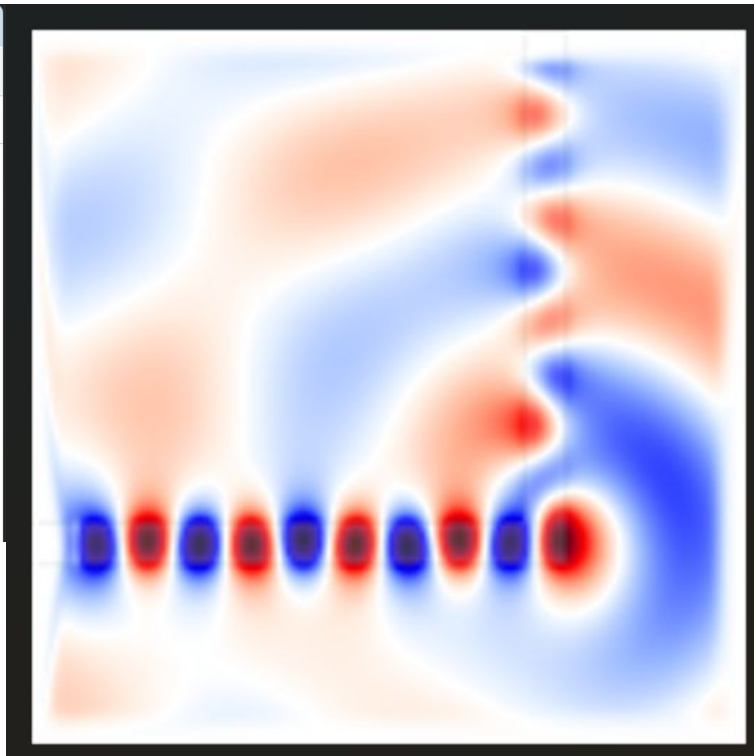
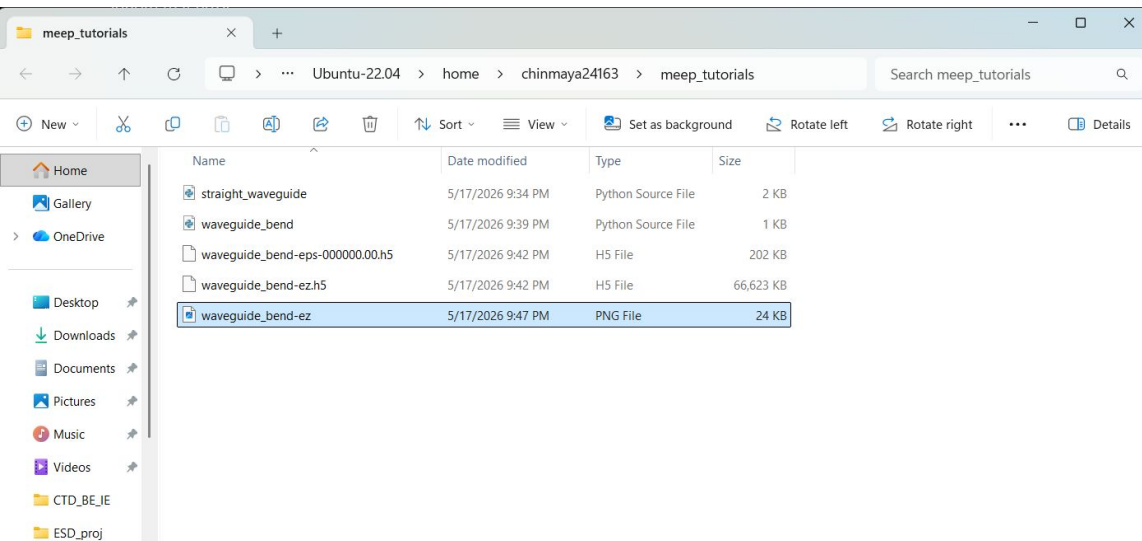
```
Elapsed run time = 0.9112 s
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ nano waveguide_bend.py
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ python waveguide_bend.py
-----
Initializing structure...
time for choose_chunkdivision = 0.000391006 s
Working in 2D dimensions.
Computational cell is 16 x 16 x 0 with resolution 10
  block, center = (-2.5,-3.5,0)
    size (12,1,1e+20)
    axes (1,0,0), (0,1,0), (0,0,1)
    dielectric constant epsilon diagonal = (12,12,12)
  block, center = (3.5,2,0)
    size (1,12,1e+20)
    axes (1,0,0), (0,1,0), (0,0,1)
    dielectric constant epsilon diagonal = (12,12,12)
time for set_epsilon = 0.0195901 s
-----
creating output file "./waveguide_bend-eps-000000.00.h5"...
creating output file "./waveguide_bend-ez.h5"...
run 0 finished at t = 200.0 (4000 timesteps)

Elapsed run time = 0.9404 s
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ ls
straight_waveguide.py waveguide_bend-eps-000000.00.h5 waveguide_bend-ez.h5 waveguide_bend.py
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ h5ls waveguide_bend-eps-000000.00.h5
eps                                Dataset {160, 160}
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ conda install -c conda-forge h5utils
2 channel Terms of Service accepted
Channels:
 - conda-forge
 - defaults
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

# All requested packages already installed.

(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ h5topng -t 332 -R -Zc dkbluered -a yarg -A waveguide_bend-eps-000000.00.h5 waveguide_bend-ez.h5
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ ls *.png
waveguide_bend-ez.png
```


Fields in a Waveguide: A 90° Bend



Fields in a Waveguide: A 90° Bend

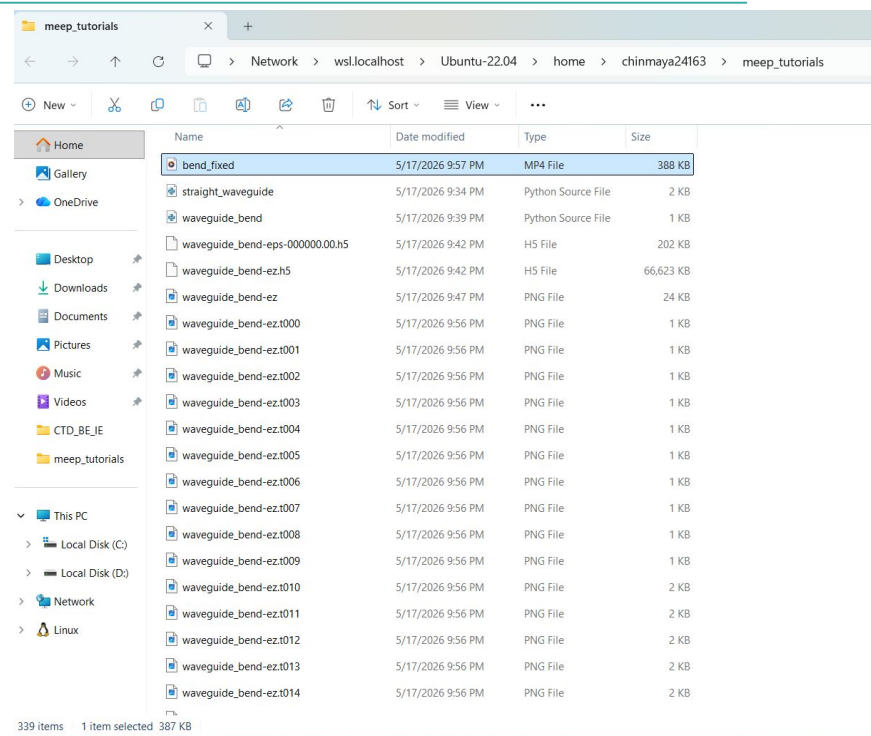


```
(mp) chinmaya24163@DESKTOP-790PGGQ:~/meep_tutorials$ hstopng -t 0:332 -R -Zc dkbluered -a yarg -A waveguide_bend-eps-000000.00.h5 waveguide_bend-ez.h5
(mp) chinmaya24163@DESKTOP-790PGGQ:~/meep_tutorials$ ffmpeg -framerate 20 -i waveguide_bend-ez.t%03d.png bend.mp4
ffmpeg version 8.1.1 Copyright (c) 2000-2026 the FFMpeg developers
built with gcc 14.3.0 (conda-forge gcc 14.3.0-18)
configuration: --prefix=/home/chinmaya24163/miniconda3/envs/mp --cc=/home/conda/feedstock_root/build_artifacts/ffmpeg-1777899865433/_build_env/bin/x86_64-conda-linux-gnu-cc --cxx=/home/conda/feedstock_root/build_artifacts/ffmpeg-1777899865433/_build_env/bin/x86_64-conda-linux-gnu-c++ --nm=/home/conda/feedstock_root/build_artifacts/ffmpeg-1777899865433/_build_env/bin/x86_64-conda-linux-gnu-nm --ar=/home/conda/feedstock_root/build_artifacts/ffmpeg-1777899865433/_build_env/bin/x86_64-conda-linux-gnu-ar --disable-doc --enable-openssl --enable-demuxer=dash --enable-hardcoded-tables --enable-libfreetype --enable-libharfbuzz --enable-libfontconfig --enable-libopenh264 --enable-libdav1d --disable-gnutls --enable-libvpx --enable-libass --enable-libplacebo --enable-pthreads --enable-alsa --enable-libpulse --enable-libvpl --enable-vaapi --enable-libopenvino --enable-gpl --enable-libx264 --enable-libx265 --enable-libmp3lame --enable-libaom --enable-libsvtav1 --enable-libxml2 --enable-pic --enable-shared --disable-static --enable-version3 --enable-zlib --enable-libvorbis --enable-libopus --enable-libsws --enable-libjxl --enable-libwebp --enable-ffplay --enable-vulkan --enable-libshaderc --pkg-config=/home/conda/feedstock_root/build_artifacts/ffmpeg-1777899865433/_build_env/bin/pkg-config
libavutil 60. 26.101 / 60. 26.101
libavcodec 62. 28.101 / 62. 28.101
libavformat 62. 12.101 / 62. 12.101
libavdevice 62. 3.101 / 62. 3.101
libavfilter 11. 14.101 / 11. 14.101
libswscale 9. 5.101 / 9. 5.101
libswresample 6. 3.101 / 6. 3.101
Input #0, image2, from 'waveguide_bend-ez.t%03d.png':
  Duration: 00:00:16.65, start: 0.000000, bitrate: N/A
  Stream #0:0: Video: png, rgb24(pc, gbr/unknown/unknown), 160x160, 20 fps, 20 tbr, 20 tbn
Stream mapping:
  Stream #0:0 -> #0:0 (png (native) -> h264 (libx264))
Press [q] to stop, [?] for help
[libx264 @ 0x575d4a77c480] using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX FMA3 BMI2 AVX2
[libx264 @ 0x575d4a77c480] profile High 4.4:4 Predictive, level 1.1, 4:4:4, 8-bit
[libx264 @ 0x575d4a77c480] 264 - core 164 r3095 baee400 - H.264/MPEG-4 AVC codec - Copyleft 2003-2022 - http://www.videolan.org/x264.html - options: cabac=1 ref=3 deblock=1:0:0 analyse=0x3:0x113 me=hex subme=7 psy=1 psy_rd=1.00:0.00 mi
xed_ref=1 me_range=16 chroma_me=1 trellis=1 8x8dct=1 cqm=0 deadzone=21,11 fast_pskip=1 chroma_qp_offset=4 threads=5 lookahead_threads=1 sliced_threads=0 nr=0 decimate=1 interlaced=0 bluray_compat=0 constrained_intra=0 bframes=3 b_pyram
id=2 b_adapt=1 b_bias=0 direct=1 weightb=1 open_gop=0 weightp=2 keyint=250 keyint_min=25 scenecut=40 intra_refresh=0 rc_lookahead=40 rc=crf mbtree=1 crf=23.0 qcomp=0.60 qpmin=0 qpmax=69 qpstep=4 ip_ratio=1.40 aq=1:1.00
Output #0, mp4, to 'bend.mp4':
  Metadata:
    encoder      : Lavf62.12.101
  Stream #0:0: Video: h264 (avc1 / 0x31637661), yuv444p(tv, progressive), 160x160, q=2-31, 20 fps, 10240 tbn
  Metadata:
    encoder      : Lavc62.28.101 libx264
  Side data:
    CPB properties: bitrate max/min/avg: 0/0/0 buffer size: 0 vbv_delay: N/A
[out#0/mp4 @ 0x575d4a77db0] video:176KiB audio:0KiB subtitle:0KiB other streams:0KiB global headers:0KiB muxing overhead: 2.503882%
frame= 333 fps=0.0 q=-1.0 Lsize= 181KiB time=00:00:16.55 bitrate= 89.3kbits/s speed= 79x elapsed=0:00:00.20
[libx264 @ 0x575d4a77c480] frame I:2 Avg QP:18.96 size: 1198
[libx264 @ 0x575d4a77c480] frame P:150 Avg QP:23.27 size: 767
[libx264 @ 0x575d4a77c480] frame B:181 Avg QP:22.79 size: 343
[libx264 @ 0x575d4a77c480] consecutive B-frames: 19.8% 19.8% 9.9% 50.5%
[libx264 @ 0x575d4a77c480] mb I I16..4: 12.0% 82.0% 6.0%
[libx264 @ 0x575d4a77c480] mb P I16..4: 3.2% 19.0% 1.7% P16..4: 21.5% 20.8% 10.9% 0.0% 0.0% skip:22.9%
[libx264 @ 0x575d4a77c480] mb B I16..4: 0.3% 1.5% 0.1% B16..8: 31.2% 9.9% 2.2% direct:3.9% skip:50.9% L0:36.2% L1:43.2% BI:20.6%
[libx264 @ 0x575d4a77c480] 8x8 transform intra:79.6% inter:97.7%
[libx264 @ 0x575d4a77c480] coded y,u,v intra: 54.2% 36.3% 33.3% inter: 17.7% 10.1% 9.4%
[libx264 @ 0x575d4a77c480] i16 v,h,d,c,p: 27% 44% 6% 23%
[libx264 @ 0x575d4a77c480] i8 v,h,d,c,ddl,ddr,vr,hd,vl,hu: 21% 21% 14% 4% 8% 5% 11% 7% 8%
[libx264 @ 0x575d4a77c480] i4 v,h,d,c,ddl,ddr,vr,hd,vl,hu: 24% 16% 13% 4% 10% 12% 7% 8% 6%
[libx264 @ 0x575d4a77c480] Weighted P-Frames: Y:23.3% UV:23.3%
[libx264 @ 0x575d4a77c480] ref P L0: 65.6% 11.6% 14.9% 7.1% 0.9%
[libx264 @ 0x575d4a77c480] ref B L0: 85.6% 5.0% 9.4%
[libx264 @ 0x575d4a77c480] ref B L1: 99.6% 0.4%
[libx264 @ 0x575d4a77c480] kb/s:86.31
```

Fields in a Waveguide: A 90° Bend



click in this area for the MP4 to start playing



Fields in a Waveguide: Transmittance Spectrum of a Waveguide Bend



```
GNU nano 6.2
# transmission around a 90-degree waveguide bend in 2d
import matplotlib.pyplot as plt
import numpy as np

import meep as mp

resolution = 10 # pixels/um

sx = 16 # size of cell in X direction
sy = 32 # size of cell in Y direction
cell = mp.Vector3(sx, sy, 0)

dpml = 1.0
pml_layers = [mp.PML(dpml)]

pad = 4 # padding distance between waveguide and cell edge
w = 1 # width of waveguide

wvg_xcen = 0.5 * (sx - w - 2 * pad) # x center of vert. wvg
wvg_ycen = -0.5 * (sy - w - 2 * pad) # y center of horiz. wvg

geometry = [
    mp.Block(
        size=mp.Vector3(mp.inf, w, mp.inf),
        center=mp.Vector3(0, wvg_ycen, 0),
        material=mp.Medium(epsilon=12),
    )
]

fcen = 0.15 # pulse center frequency
df = 0.1 # pulse width (in frequency)
sources = [
    mp.Source(
        mp.GaussianSource(fcen, fwidth=df),
        component=mp.Ez,
        center=mp.Vector3(-0.5 * sx + dpml, wvg_ycen, 0),
        size=mp.Vector3(0, w, 0),
    )
]

sim = mp.Simulation(
    cell_size=cell,
    boundary_layers=pml_layers,
    geometry=geometry,
    sources=sources,
    resolution=resolution,
)

nfreq = 100 # number of frequencies at which to compute flux

# reflected flux
refl_fr = mp.FluxRegion(
    center=mp.Vector3(-0.5 * sx + dpml + 0.5, wvg_ycen, 0), size=mp.Vector3(0, 2 * w, 0)
)
refl = sim.add_flux(fcen, df, nfreq, refl_fr)

# transmitted flux
tran_fr = mp.FluxRegion(
    center=mp.Vector3(0.5 * sx - dpml, wvg_ycen, 0), size=mp.Vector3(0, 2 * w, 0)
)
```

```
GNU nano 6.2
)
tran = sim.add_flux(fcen, df, nfreq, tran_fr)

pt = mp.Vector3(0.5 * sx - dpml - 0.5, wvg_ycen)

sim.run(until_after_sources=mp.stop_when_fields_decayed(50, mp.Ez, pt, 1e-3))

# for normalization run, save flux fields data for reflection plane
straight_refl_data = sim.get_flux_data(refl)

# save incident power for transmission plane
straight_tran_flux = mp.get_fluxes(tran)

sim.reset_meep()

geometry = [
    mp.Block(
        mp.Vector3(sx - pad, w, mp.inf),
        center=mp.Vector3(-0.5 * pad, wvg_ycen),
        material=mp.Medium(epsilon=12),
    ),
    mp.Block(
        mp.Vector3(w, sy - pad, mp.inf),
        center=mp.Vector3(wvg_xcen, 0.5 * pad),
        material=mp.Medium(epsilon=12),
    ),
]

sim = mp.Simulation(
    cell_size=cell,
    boundary_layers=pml_layers,
    geometry=geometry,
    sources=sources,
    resolution=resolution,
)

# reflected flux
refl = sim.add_flux(fcen, df, nfreq, refl_fr)

tran_fr = mp.FluxRegion(
    center=mp.Vector3(wvg_xcen, 0.5 * sy - dpml - 0.5, 0), size=mp.Vector3(2 * w, 0, 0)
)
tran = sim.add_flux(fcen, df, nfreq, tran_fr)

# for normal run, load negated fields to subtract incident from refl. fields
sim.load_minus_flux_data(refl, straight_refl_data)

pt = mp.Vector3(wvg_xcen, 0.5 * sy - dpml - 0.5)

sim.run(until_after_sources=mp.stop_when_fields_decayed(50, mp.Ez, pt, 1e-3))

bend_refl_flux = mp.get_fluxes(refl)
bend_tran_flux = mp.get_fluxes(tran)

flux_freqs = mp.get_flux_freqs(refl)

wl = []
Rs = []
Ts = []
for i in range(nfreq):
```

```
    wl = []
    Rs = []
    Ts = []
    for i in range(nfreq):
        wl = np.append(wl, 1 / flux_freqs[i])
        Rs = np.append(Rs, -bend_refl_flux[i] / straight_tran_flux[i])
        Ts = np.append(Ts, bend_tran_flux[i] / straight_tran_flux[i])

if mp.am_master():
    plt.figure()
    plt.plot(wl, Rs, "bo-", label="reflectance")
    plt.plot(wl, Ts, "ro-", label="transmittance")
    plt.plot(wl, 1 - Rs - Ts, "go-", label="loss")
    plt.axis([5.0, 10.0, 0, 1])
    plt.xlabel("wavelength (um)")
    plt.legend(loc="upper right")
    plt.show()
```

Fields in a Waveguide: Transmittance Spectrum of a Waveguide Bend



```
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ python transmittance.py
```

```
-----  
Initializing structure...
```

```
time for choose_chunkdivision = 0.000239849 s
```

```
Working in 2D dimensions.
```

```
Computational cell is 16 x 32 x 0 with resolution 10
```

```
block, center = (0,-11.5,0)
```

```
size (1e+20,1,1e+20)
```

```
axes (1,0,0), (0,1,0), (0,0,1)
```

```
dielectric constant epsilon diagonal = (12,12,12)
```

```
time for set_epsilon = 0.033946 s
```

```
-----  
field decay(t = 50.050000000000004): 4.8251893805577874e-09 / 4.8251893805577874e-09 = 1.0  
field decay(t = 100.050000000000001): 0.02880180987942579 / 0.02880180987942579 = 1.0
```

```
field decay(t = 150.1): 0.026893465093385728 / 0.02880180987942579 = 0.9337421921042793
```

```
field decay(t = 200.15): 2.3158397341290867e-13 / 0.02880180987942579 = 8.040604891928607e-12
```

```
run 0 finished at t = 200.15 (4003 timesteps)
```

```
-----
```

```
Initializing structure...
```

```
time for choose_chunkdivision = 0.000115156 s
```

```
Working in 2D dimensions.
```

```
Computational cell is 16 x 32 x 0 with resolution 10
```

```
block, center = (-2,-11.5,0)
```

```
size (12,1,1e+20)
```

```
axes (1,0,0), (0,1,0), (0,0,1)
```

```
dielectric constant epsilon diagonal = (12,12,12)
```

```
block, center = (3.5,2,0)
```

```
size (1,20,1e+20)
```

```
axes (1,0,0), (0,1,0), (0,0,1)
```

```
dielectric constant epsilon diagonal = (12,12,12)
```

```
time for set_epsilon = 0.036119 s
```

```
-----  
field decay(t = 50.050000000000004): 1.6976522694449e-10 / 1.6976522694449e-10 = 1.0
```

```
field decay(t = 100.050000000000001): 4.691071063910552e-07 / 4.691071063910552e-07 = 1.0
```

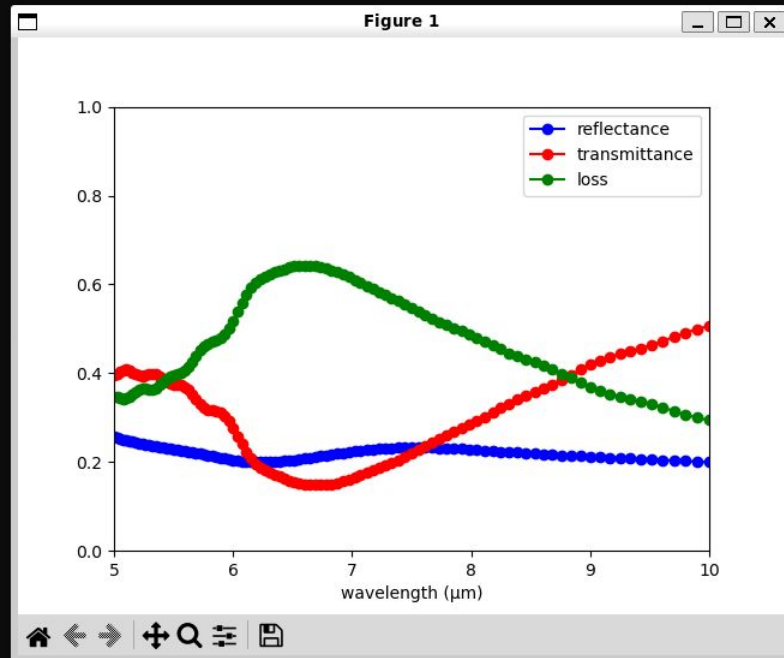
```
field decay(t = 150.1): 2.9928727336864534e-07 / 4.691071063910552e-07 = 0.6379934758847049
```

```
field decay(t = 200.15): 0.003927813565272275 / 0.003927813565272275 = 1.0
```

```
field decay(t = 250.200000000000002): 0.00015009081939073958 / 0.003927813565272275 = 0.03821230740627969
```

```
field decay(t = 300.2): 8.80622639574523e-11 / 0.003927813565272275 = 2.2420174097888442e-08
```

```
run 0 finished at t = 300.2 (6004 timesteps)
```



Angular Reflectance Spectrum of a Planar Interface



```
GNU nano 6.2
import argparse
import math

import mup as mp

def main(args):
    resolution = args.res

    dpml = 1.0 # PML thickness
    sz = 10 + 2 * dpml # size of computational cell (without PMLs)
    cell_size = mp.Vector3(0, 0, sz)
    pml_layers = [mp.PML(dpml)]

    wvl_min = 0.4 # min wavelength
    wvl_max = 0.8 # max wavelength
    fmin = 1 / wvl_max # min frequency
    fmax = 1 / wvl_min # max frequency
    fcen = 0.5 * (fmin + fmax) # center frequency
    df = fmax - fmin # frequency width
    nfreq = 50 # number of frequency bins

    # rotation angle (in degrees) of source: CCW around Y axis, 0 degrees along +Z axis
    theta_r = math.radians(args.theta)

    # plane of incidence is XZ
    k = mp.Vector3(math.sin(theta_r), 0, math.cos(theta_r)).scale(fmin)

    # if normal incidence, force number of dimensions to be 1
    dimensions = 1 if theta_r == 0 else 3
    sources = [
        mp.Source(
            mp.GaussianSource(fcen, fwidth=df),
            component=mp.Ex,
            center=mp.Vector3(0, 0, -0.5 * sz + dpml),
        )
    ]

    sim = mp.Simulation(
        cell_size=cell_size,
        boundary_layers=pml_layers,
        sources=sources,
        k_point=k,
        dimensions=dimensions,
        resolution=resolution,
    )

    refl_fr = mp.FluxRegion(center=mp.Vector3(0, 0, -0.25 * sz))
    refl = sim.add_flux(fcen, df, nfreq, refl_fr)

    sim.run(
        until_after_sources=mp.stop_when_fields_decayed(
            50, mp.Ex, mp.Vector3(0, 0, -0.5 * sz + dpml), 1e-9
        )
    )

    empty_flux = mp.get_fluxes(refl)
    empty_data = sim.get_flux_data(refl)
    sim.reset_meeq()
```

```
GNU nano 6.2 reflect
    until_after_sources=mp.stop_when_fields_decayed(
        50, mp.Ex, mp.Vector3(0, 0, -0.5 * sz + dpml), 1e-9
    )
)

empty_flux = mp.get_fluxes(refl)
empty_data = sim.get_flux_data(refl)
sim.reset_meeq()

# add a block with n=3.5 for the air-dielectric interface
geometry = [
    mp.Block(
        size=mp.Vector3(mp.inf, mp.inf, 0.5 * sz),
        center=mp.Vector3(0, 0, 0.25 * sz),
        material=mp.Medium(index=3.5),
    )
]

sim = mp.Simulation(
    cell_size=cell_size,
    geometry=geometry,
    boundary_layers=pml_layers,
    sources=sources,
    k_point=k,
    dimensions=dimensions,
    resolution=resolution,
)

refl = sim.add_flux(fcen, df, nfreq, refl_fr)
sim.load_minus_flux_data(refl, empty_data)

sim.run(
    until_after_sources=mp.stop_when_fields_decayed(
        50, mp.Ex, mp.Vector3(0, 0, -0.5 * sz + dpml), 1e-9
    )
)

refl_flux = mp.get_fluxes(refl)
freqs = mp.get_flux_freqs(refl)

for i in range(nfreq):
    print(
        f"refl: {k.x}, {1 / freqs[i]}, {math.degrees(math.asin(k.x/freqs[i]))}, {-refl_flux[i] / empty_flux[i]}"
    )

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "-res", type=int, default=200, help="resolution (default: 200 pixels/um)"
    )
    parser.add_argument(
        "-theta",
        type=float,
        default=0,
        help="angle of incident planewave (default: 0 degrees)",
    )
    args = parser.parse_args()
    main(args)
```

Angular Reflectance Spectrum of a Planar Interface



Creating an
automation
shell script

```
GNU nano 6.2
#!/bin/bash

for i in `seq 0 5 80`; do
    python -u reflectance.py -theta $i | tee flux_t${i}.out
    grep refl: flux_t${i}.out | cut -d , -f2- > flux_t${i}.dat
done
```

Angular Reflectance Spectrum of a Planar Interface



Making
Executable and
running it

```
(mp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ chmod +x run_angles.sh
(mmp) chinmaya24163@DESKTOP-79DPGGQ:~/meep_tutorials$ ./run_angles.sh
-----
Initializing structure...
time for choose_chunkdivision = 0.000305891 s
Working in 1D dimensions.
Computational cell is 0 x 0 x 12 with resolution 200
time for set_epsilon = 0.000290155 s
-----
Meep: using complex fields.
field decay(t = 50.0025): 0.2502050785798136 / 0.2502050785798136 = 1.0
field decay(t = 100.0025): 1.0474120541054926e-14 / 0.2502050785798136 = 4.18621420496617e-14
run 0 finished at t = 100.0025 (40001 timesteps)
-----
Initializing structure...
time for choose_chunkdivision = 7.29561e-05 s
Working in 1D dimensions.
Computational cell is 0 x 0 x 12 with resolution 200
block, center = (0,0,3)
size (1e+20,1e+20,6)
axes (1,0,0), (0,1,0), (0,0,1)
dielectric constant epsilon diagonal = (12.25,12.25,12.25)
time for set_epsilon = 0.000434875 s
-----
Meep: using complex fields.
field decay(t = 50.0025): 0.25020507848496115 / 0.25020507848496115 = 1.0
field decay(t = 100.0025): 2.8160152443947816e-13 / 0.25020507848496115 = 1.1254020484882418e-12
run 0 finished at t = 100.0025 (40001 timesteps)
refl.: 0.0, 0.8, 0.0, 0.30796135893943605
refl.: 0.0, 0.784, 0.0, 0.3078435119619084
refl.: 0.0, 0.76862745098083922, 0.0, 0.30775722165660886
refl.: 0.0, 0.75304615330461539, 0.0, 0.307665637670457753
refl.: 0.0, 0.739622641509434, 0.0, 0.30762610323503883
refl.: 0.0, 0.7259259259259259, 0.0, 0.30760279133006696
refl.: 0.0, 0.7127272727272727, 0.0, 0.30758914545291427
refl.: 0.0, 0.7, 0.0, 0.30756147371040887
refl.: 0.0, 0.6877192982456141, 0.0, 0.30751713545942894
refl.: 0.0, 0.6758620689655173, 0.0, 0.30747627387135945
refl.: 0.0, 0.664060779661017, 0.0, 0.30743041586190694
refl.: 0.0, 0.6533333333333333, 0.0, 0.30738928065298266
refl.: 0.0, 0.6426229508196721, 0.0, 0.30734550115741993
refl.: 0.0, 0.632258064516129, 0.0, 0.30730104402509106
refl.: 0.0, 0.6222222222222222, 0.0, 0.3072570396798615
refl.: 0.0, 0.6124999999999999, 0.0, 0.30721501543863605
refl.: 0.0, 0.6030769230769231, 0.0, 0.3071708675327489
refl.: 0.0, 0.593939393939394, 0.0, 0.3071238578607356
refl.: 0.0, 0.5850746269656717, 0.0, 0.30707811626390247
refl.: 0.0, 0.5768705802352942, 0.0, 0.30703137700253233
refl.: 0.0, 0.5681159420289855, 0.0, 0.30698360479202014
refl.: 0.0, 0.56, 0.0, 0.3069329118885353
refl.: 0.0, 0.552112676056338, 0.0, 0.3068832828831594
refl.: 0.0, 0.5444444444444444, 0.0, 0.30683276581026087
refl.: 0.0, 0.536986301369863, 0.0, 0.3067825234908616
refl.: 0.0, 0.5297297297297298, 0.0, 0.3067320512659318
refl.: 0.0, 0.5226666666666667, 0.0, 0.3066807204499778
refl.: 0.0, 0.5157894736842105, 0.0, 0.3066273334438513
refl.: 0.0, 0.5090909090909091, 0.0, 0.30657290003665555
refl.: 0.0, 0.5025641025641026, 0.0, 0.3065176574925483
refl.: 0.0, 0.4962025316455696, 0.0, 0.3064616465758423
refl.: 0.0, 0.49, 0.0, 0.30640602153357155
refl.: 0.0, 0.4839506172839506, 0.0, 0.3063503143363787
refl.: 0.0, 0.478047804780486, 0.0, 0.30629397180729357
refl.: 0.0, 0.4722015562650605, 0.0, 0.3062365265393308
refl.: 0.0, 0.4666666666666667, 0.0, 0.3061780851314222,
```


Angular Reflectance Spectrum of a Planar Interface



Creating plotting script

```
GNU nano 6.2
import matplotlib.pyplot as plt
import numpy as np
import numpy.matlib
import math

theta_in = np.arange(0,85,5)

kxs = np.empty((50,theta_in.size))
thetas = np.empty((50,theta_in.size))
Rmeep = np.empty((50,theta_in.size))

for j in range(theta_in.size):
    f = np.genfromtxt(f"flux_t{theta_in[j]}.dat", delimiter=",")

    kxs[:,j] = f[:,0]
    thetas[:,j] = f[:,2]
    Rmeep[:,j] = f[:,3]

wvl = f[:,1]

wvls = np.matlib.repmat(np.reshape(wvl,(wvl.size,1)),1,theta_in.size)

n1 = 1
n2 = 3.5

theta_out = lambda theta_in: math.asin(n1*math.sin(theta_in)/n2)

Rfresnel = lambda theta_in: abs(
    (
        n1*math.cos(theta_out(theta_in))
        - n2*math.cos(theta_in)
    )
    /
    (
        n1*math.cos(theta_out(theta_in))
        + n2*math.cos(theta_in)
    )
)**2

Ranalytic = np.empty((50,theta_in.size))

for m in range(wvl.size):
    for n in range(theta_in.size):
        Ranalytic[m,n] = Rfresnel(math.radians(thetas[m,n]))

fig, axs = plt.subplots(2,2, figsize=(10,7))

wvl_idx = 0

axs[0,0].plot(theta_in,Rmeep[wvl_idx,:], 'bo-', label='meep')
axs[0,0].plot(theta_in,Ranalytic[wvl_idx,:], 'ro-', label='analytic')

axs[0,0].set_xlabel("incident angle (degrees)")
axs[0,0].set_ylabel(f"reflectance at λ={wvl[wvl_idx]:.2f} μm")
axs[0,0].legend()

pc = axs[0,1].pcolormesh(
    kxs,
    wvls,
    Rmeep,
```

```
)

axs[0,1].set_xlabel("kx")
axs[0,1].set_ylabel("wavelength (μm)")
axs[0,1].set_title("Reflectance (Meep)")

fig.colorbar(pc, ax=axs[0,1])

pc = axs[1,0].pcolormesh(
    thetas,
    wvls,
    Rmeep,
    cmap='hot_r',
    shading='gouraud'
)

axs[1,0].set_xlabel("angle (degrees)")
axs[1,0].set_ylabel("wavelength (μm)")
axs[1,0].set_title("Reflectance vs angle")

fig.colorbar(pc, ax=axs[1,0])

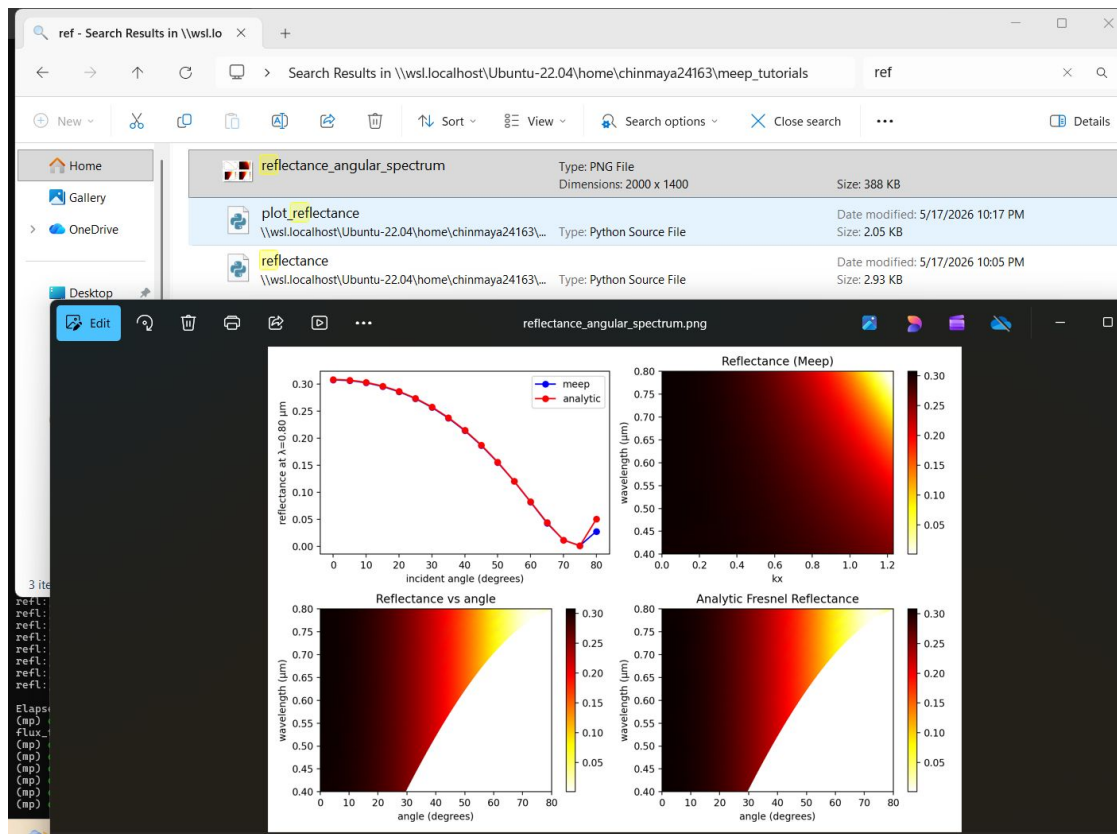
pc = axs[1,1].pcolormesh(
    thetas,
    wvls,
    Ranalytic,
    cmap='hot_r',
    shading='gouraud'
)

axs[1,1].set_xlabel("angle (degrees)")
axs[1,1].set_ylabel("wavelength (μm)")
axs[1,1].set_title("Analytic Fresnel Reflectance")

fig.colorbar(pc, ax=axs[1,1])

plt.tight_layout()

plt.savefig("reflectance_angular_spectrum.png", dpi=200)
```



Bibliography



- <https://github.com/NanoComp/meep/blob/master/python/examples/straight-waveguide.py>
- <https://github.com/NanoComp/meep/blob/master/python/examples/bent-waveguide.py>
- <https://github.com/NanoComp/meep/blob/master/python/examples/bend-flux.py>
- <https://github.com/NanoComp/meep/blob/master/python/examples/refl-angular.p>
- https://meep.readthedocs.io/en/master/Python_Tutorials/Basics/#the-meep-library